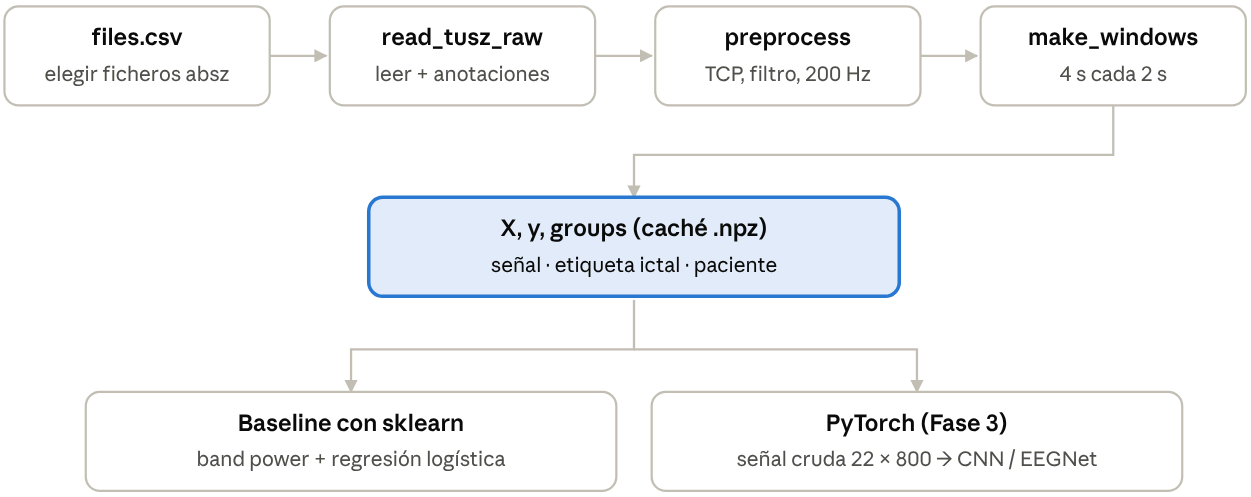
Guía 09 · De MNE a Machine Learning

Oct 15, 2026 · @Daniel Gómez

Objetivo y pipeline

Este módulo junta todo el curso en un **pipeline reproducible**: recorre varios ficheros de TUSZ, produce `X`, `y` y `groups` (paciente), entrena un baseline evaluado sin fuga de pacientes y exporta los datos para PyTorch. Es el puente directo a la Fase 3. Resultado: con 14 ficheros de ausencias, una regresión logística sobre potencia por bandas logra **ROC-AUC 0,79 y PR-AUC 0,29** (azar = 0,04).

Qué	Valor
Notebook	`Curso_MNE/09_De_MNE_a_ML.ipynb`
Fase del TFM	Fase 3 · Modelo (baseline)
Duración	~2 h
Datos	Todas las crisis `absz` de *train* con montaje LE o AR
Salida	`salidas/dataset_absz_train/` con `X.npy`, `y.npy`, `meta.csv`



El pipeline completo del módulo 09

Cada caja es una función de `tusz_utils.py` que construiste en los módulos anteriores. Lo nuevo aquí es encadenarlas para muchos ficheros, guardar en caché y evaluar bien.

Construir el dataset multi-fichero

9.1 · Elegir ficheros con el inventario. Se filtra `files.csv`: split `train`, etiqueta que contenga `absz` y montaje `01_tcp_ar` o `02_tcp_le` (los que dan 22 canales TCP). Salen **14 ficheros de 8 pacientes**, todos a 250 Hz y de 4 a 25 minutos.

9.2 · Extraer un fichero. `extract(file_id, patient)` encadena `read_tusz_raw` → `preprocess` → `make_windows` → `drop_bad(reject=1500e-6)` y añade a la metadata el paciente y el fichero, imprescindibles para validar. El umbral de 1 500 µV es deliberadamente alto: solo quita basura extrema, porque la Guía 06 demostró que umbrales bajos borran ausencias.

9.3 · Recorrer todo con caché. Cada fichero procesado se guarda en `salidas/cache_absz/<fichero>.npz`. Si ya existe, se lee en lugar de recalcularse, así que puedes relanzar el notebook sin esperar. Los errores (fichero no descargado, canales raros) se registran y el bucle sigue. Resultado: 14 ficheros en 76 s, 0 errores.

Resultado	Valor
`X`	`(7 532, 22, 800)`, float32, 0,53 GB

Resultado	Valor
Proporción ictal	4,14 % (312 ventanas)
Pacientes	8

Paciente	Ventanas ictales	Ventanas interictales
aaaaadqe	76	1 008
aaaaabqv	67	1 335
aaaaaccj	62	1 855
aaaaadyf	53	281
aaaaacrb	21	596
aaaaamtc	12	752
aaaaaasy	11	669
aaaaaenl	10	724

Lo que te dice esta tabla: los pacientes aportan cantidades muy distintas (de 10 a 76 ventanas ictales). Si un paciente domina el entrenamiento, el modelo aprenderá sobre todo *sus* crisis. Es otra razón para validar por paciente.

Validación por paciente y baseline

Regla de oro de la validación en EEG: separar por paciente. Si ventanas del mismo paciente caen en entrenamiento y en test, el modelo aprende a reconocer *al paciente* (su forma de onda, su equipo, sus artefactos) y no la crisis, y las métricas se inflan. TUSZ ya separa pacientes entre *train*, *dev* y *eval*; dentro de *train* se usa `GroupKFold` con `groups = paciente`.

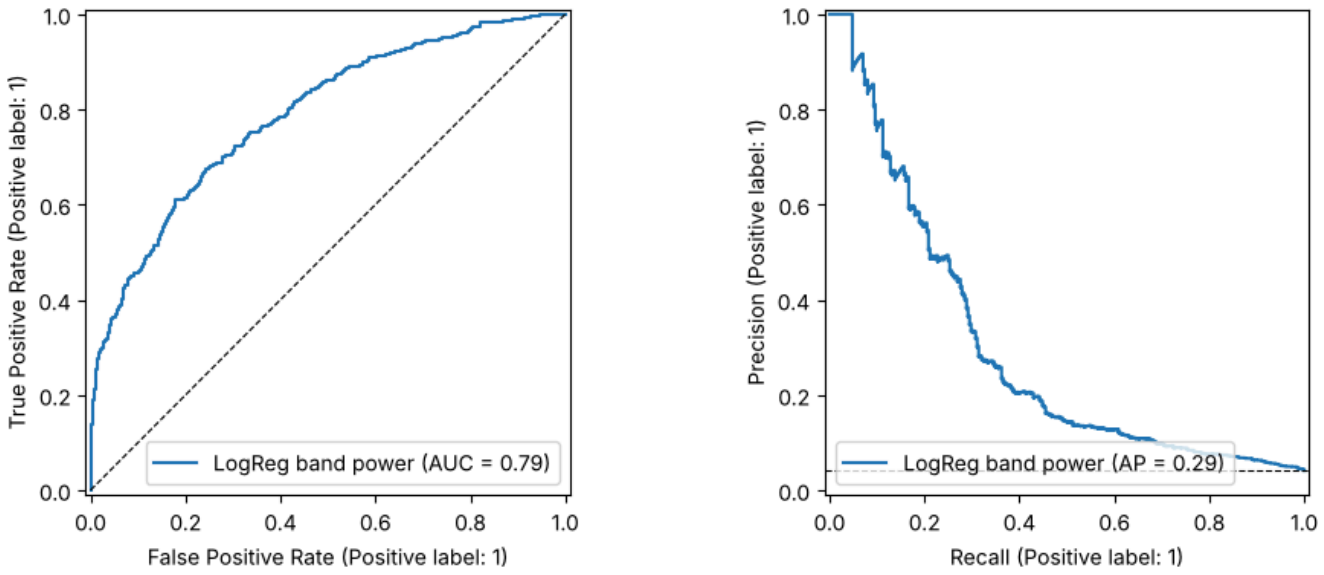
```
feats = band_power(mne.EpochsArray(X * 1e-6, info), relative=True, log=True)
F = feats.reshape(len(feats), -1)                       # (7 532, 110) = 22
canales × 5 bandas
clf = make_pipeline(StandardScaler(),
                    LogisticRegression(class_weight="balanced",
max_iter=2000))
cv = GroupKFold(n_splits=5)
proba = cross_val_predict(clf, F, y, groups=groups, cv=cv,
method="predict_proba")[:, 1]
```

`cross_val_predict` da para cada ventana la probabilidad que predijo el modelo **cuando ese paciente no estaba en el entrenamiento**. Así todas las ventanas tienen una predicción honesta.

Resultados (5 pliegues por paciente)

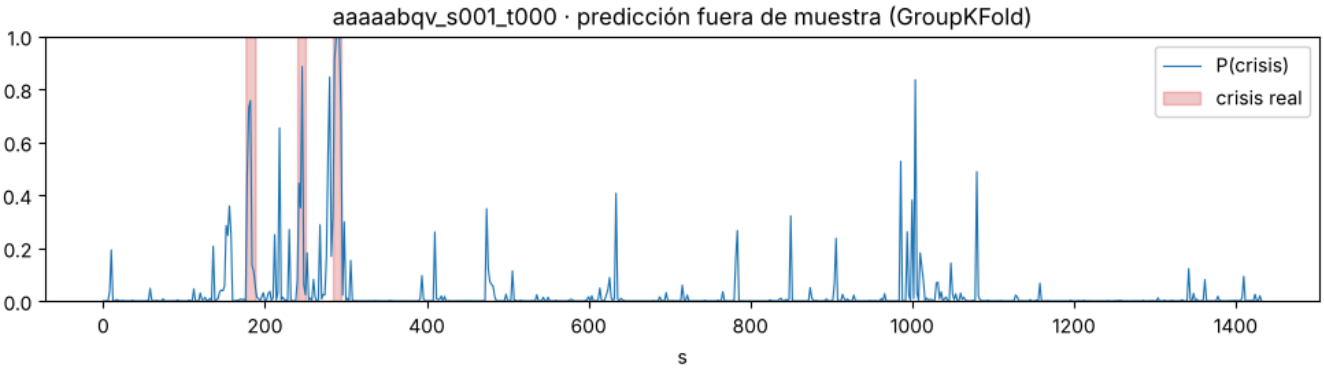
Modelo	ROC-AUC	PR-AUC	Comentario
Azar	0,50	0,041	Proporción de ictales
LogReg + potencia por bandas (110 features)	0,789	0,294	Baseline principal
LogReg + PSD multitaper completo (`mne.decoding`)	0,687	0,284	Más features, peor ROC
Random Forest + potencia por bandas	**0,817**	**0,375**	El mejor (E9.1)

Con umbral 0,5, la regresión logística detecta el **66,7 %** de las ventanas de crisis (*recall*), pero solo el **10,9 %** de sus alarmas son crisis (*precision*). La accuracy es 0,76 y la *balanced accuracy* 0,715.



Cómo leerla. ROC (izquierda): cada punto es un umbral; arriba a la izquierda es perfecto, la diagonal es azar. AUC 0,79 = el modelo ordena bien una ventana ictal por encima de una no ictal el 79 % de las veces. **PR** (derecha): precisión frente a sensibilidad. La línea discontinua de abajo es el azar (0,04). Para captar más del 60 % de las ventanas de crisis, la precisión cae por debajo del 20 %.

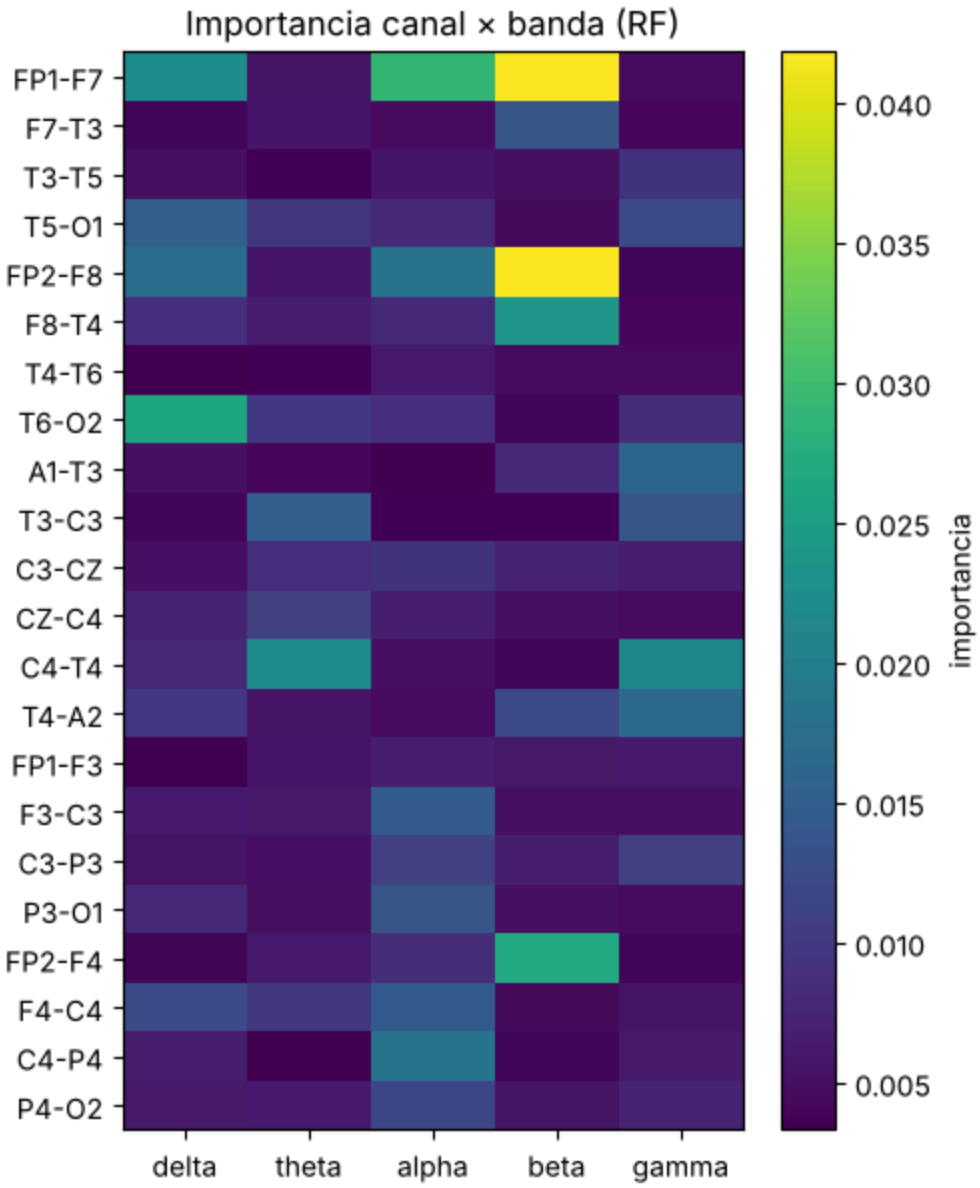
Por qué mirar PR-AUC y no accuracy. Con 4 % de crisis, la accuracy premia decir siempre "no". La ROC también puede parecer optimista con clases muy desbalanceadas. **PR-AUC se centra en la clase rara**, que es la que te importa.



Cómo leerla: la vista clínica. Las tres crisis reales (rojo, ~180-290 s) tienen picos de probabilidad altos. Pero hay muchos otros picos sueltos, algunos por encima de 0,8 hacia 1 000 s. Esos son **falsos positivos**, y varios coinciden con los artefactos que viste en el espectrograma de la Guía 08. Esta figura, con el espectrograma encima, es tu futura figura "espectrograma + predicciones".

Primera explicabilidad y métricas por evento

E9.1 · Random Forest y sus importancias. Cambiar la regresión logística por un `RandomForestClassifier(class_weight="balanced")` mejora a ROC-AUC 0,817 y PR-AUC 0,375. Además, `feature_importances_` reorganizado como 22 canales × 5 bandas dice en qué se fija el modelo. Es tu **primer contacto con la XAI**.



Cómo leerla. Cada casilla es la importancia de una banda en un canal; amarillo = más importante. Destacan **beta en FP1-F7 y FP2-F8**, seguidas de alfa en FP1-F7 y delta en T6-O2. Interpretación con cautela:

- Esperabas delta (la punta-onda). El modelo la usa, pero prioriza **beta en canales frontopolares**, la zona de parpadeos y músculo frontal.

- Puede ser real (las puntas tienen energía en alta frecuencia, Guía 08) o un **atajo**: el modelo aprende artefactos que acompañan a las crisis en estos pacientes.
- Con 8 pacientes no puedes distinguirlo. Es justo el tipo de pregunta que responderá la fase de XAI (SHAP, Grad-CAM) comparando con las anotaciones por canal (Guía 04).

E9.2 · Métricas por evento. En clínica no importa cada ventana, sino si se detectó *la crisis* y cuántas falsas alarmas hay por hora.

Métrica	Resultado	Lectura
Sensibilidad por evento (umbral 0,5)	47 / 47 = **1,00**	Todas las crisis tienen al menos una ventana con P > 0,5
Falsos positivos	1 709 ventanas → **~408 FP/hora**	Inaceptable clínicamente

La lección más importante del curso. Una sensibilidad perfecta no vale nada con 408 falsas alarmas por hora. Las cifras de referencia en la literatura de TUSZ se miden en unos pocos FP cada 24 horas. Para acercarte: fusionar ventanas contiguas en eventos, exigir varias ventanas seguidas por encima del umbral, subir el umbral y, sobre todo, un modelo mejor (la red de la Fase 3). Reporta **siempre ambas cifras juntas**.

E9.3 · Test externo con *dev*. El notebook deja aquí un esqueleto (no es una celda ejecutable): entrenar con todo *train* y evaluar en los ficheros `absz` de *dev* (4 pacientes nuevos). Así es como reportarás los resultados en la memoria, y es tu siguiente paso natural.

Exportar a PyTorch y próximos pasos

Formato recomendado: un `.npy` por *split* con la señal y las etiquetas. Se puede abrir con `mmap_mode="r"` para no cargarlo entero en RAM cuando crezca.

```
OUT = Path("salidas/dataset_absz_train")
np.save(OUT / "X.npy", X)          # (7 532, 22, 800) float32, µV
np.save(OUT / "y.npy", y)
meta.to_csv(OUT / "meta.csv", index=False)
```

```
class TUSZWindows(Dataset):
    def __init__(self, folder):
        self.X = np.load(f"{folder}/X.npy", mmap_mode="r")
        self.y = np.load(f"{folder}/y.npy")
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        x = torch.from_numpy(np.array(self.X[i]))
        x = (x - x.mean(-1, keepdim=True)) / (x.std(-1, keepdim=True) + 1e-6)
# z-score por canal
        return x, torch.tensor(self.y[i], dtype=torch.float32)

dl = DataLoader(TUSZWindows("salidas/dataset_absz_train"), batch_size=64,
shuffle=True)
xb, yb = next(iter(dl))     # xb: (64, 22, 800) → entrada de una CNN 1D /
EEGNet
```

Cuándo usar cada métrica

Métrica	Úsala para	Cuidado
ROC-AUC	Comparar modelos en general	Optimista con clases muy desbalanceadas
PR-AUC	Tu métrica principal por ventana	Compárala siempre con el azar (proporción de positivos)
Balanced accuracy	Un número legible con umbral fijo	Depende del umbral
Sensibilidad por evento	Lo que importa en clínica	Sin FP/hora no significa nada
FP por hora	Utilidad real de una alarma	Fusiona ventanas contiguas antes
Accuracy	Casi nunca	Premia decir siempre “no crisis”

Próximos pasos naturales

1. Ampliar `sel` a todos los tipos de crisis y a *dev* → dataset de detección completo.
2. Encadenar los `t00x` de cada sesión para tener más contexto preictal (predicción) y corregir el etiquetado del SPH (Guía 07).

3. Sustituir el baseline por una red (CNN 1D / EEGNet) leyendo `X.npy` con el `Dataset` de arriba.

Autoevaluación

- [ ] ¿Por qué `GroupKFold` por paciente? → Para que el modelo no aprenda al paciente en vez de la crisis.
- [ ] ¿Qué PR-AUC tendría un modelo al azar aquí? → 0,041, la proporción de ictales.
- [ ] ¿Qué dos cifras reportas siempre juntas? → Sensibilidad por evento y FP por hora.
- [ ] ¿Por qué el umbral de rechazo es 1 500 µV? → Umbrales bajos borran ausencias (Guía 06).
- [ ] ¿Qué pregunta abre el mapa de importancias? → Si el modelo usa la crisis o artefactos frontales.

Fin del curso. Con esto tienes todo lo de MNE para las Fases 1-2 del TFM y el puente a la Fase 3.